Modelo Entidad-Relación: Sistema de Gestión Académica de una Universidad

Ejercicio resuelto con fundamentos, esquemas y desarrollo paso a paso Asignatura: Bases de Datos · Nivel: pregrado · Tema: modelado conceptual y paso al modelo relacional

1. Qué vamos a resolver

El ejercicio pide diseñar la base de datos de una universidad que administra estudiantes, profesores, programas, cursos, grupos, matrículas y planes de estudio. El resultado debe ser un **Modelo Entidad-Relación (MER)** normalizado, con **7 tablas**, y las cardinalidades correctas entre ellas.

Antes de dibujar nada conviene tener claros los conceptos de base. Los repaso primero, luego muestro los esquemas y al final resuelvo el ejercicio en el orden en que lo haría un diseñador: de las palabras del enunciado a las tablas listas para crear.

2. Fundamentos teóricos

2.1 Origen y propósito del modelo

El MER lo propuso **Peter Chen en 1976** en su artículo *The Entity-*

Relationship Model: Toward a Unified View of Data. La idea era dar a analistas y usuarios un lenguaje común para describir los datos de un sistema **sin pensar todavía en tablas, índices ni motores**. Por eso se le llama modelo *conceptual*: describe el negocio, no la implementación.

El flujo clásico de diseño tiene tres etapas:

1. **Modelo conceptual:** el MER, independiente del gestor.
2. **Modelo lógico:** el esquema relacional (tablas, claves, restricciones).
3. **Modelo físico:** tipos de dato, índices, particiones, ajustes del motor.

Este documento cubre las dos primeras y deja la tercera esbozada con SQL.

2.2 Elementos del modelo

Entidad. Cualquier cosa del mundo real que podamos distinguir y de la que queramos guardar datos: un estudiante, un curso, un profesor. En el diagrama se dibuja como rectángulo.

Atributo. Una propiedad de la entidad. Los hay de varios tipos:

- **Simple o compuesto:** `nombre` frente a `dirección` (calle, ciudad, barrio).
- **Monovalorado o multivalorado:** un solo correo frente a varios teléfonos.
- **Derivado:** se calcula, no se guarda (la edad sale de `fecha_nacimiento`).
- **Clave:** identifica de forma única cada instancia de la entidad.

Relación. La asociación entre dos o más entidades, casi siempre expresada con un verbo: el profesor *dicta* un grupo, el estudiante *realiza* una matrícula. Se dibuja como rombo.

Grado. Cuántas entidades participan: binaria (dos), ternaria (tres) o n-aria. Todas las relaciones de este ejercicio son binarias.

2.3 Cardinalidad y participación

Son las dos restricciones que más errores producen en clase, así que vale la pena separarlas.

La **cardinalidad** responde a *cuántos*. Cuántas instancias de A se asocian con una de B:

Tipo	Lectura	Ejemplo
1:1	Una con una	Programa ↔ Plan de estudio
1:N	Una con muchas	Programa → Cursos
N:M	Muchas con muchas	Estudiantes ↔ Grupos

La **participación** responde a *si es obligatorio*. Es **total** cuando toda instancia debe estar relacionada (un grupo no puede existir sin curso) y **parcial** cuando puede quedar suelta (un profesor recién contratado todavía sin grupos).

2.4 Entidad asociativa

Una relación N:M no se puede representar directamente en el modelo relacional. Se resuelve con una **tabla intermedia** que guarda las claves de ambos lados. Cuando esa tabla además carga datos propios, como la fecha de matrícula o la nota final, deja de ser un simple puente y se modela como una **entidad asociativa**. En este ejercicio, `MATRÍCULA` cumple justo ese papel.

2.5 Del MER al modelo relacional

Las reglas de transformación que usaremos:

Elemento del MER	Resultado relacional
Entidad	Una tabla con su clave primaria
Relación 1:N	Clave foránea en la tabla del lado N

Relación 1:1	Clave foránea con restricción `UNIQUE` en uno de los lados
Relación N:M	Tabla intermedia con dos claves foráneas
Atributo multivalorado	Tabla aparte

2.6 Normalización

Normalizar es organizar las tablas para que cada dato viva en un solo lugar. Para este ejercicio basta recordar tres niveles:

- **1FN:** valores atómicos, sin listas ni grupos repetidos en una celda.
- **2FN:** cada atributo depende de **toda** la clave, no de una parte.
- **3FN:** ningún atributo depende de otro atributo que no sea clave.

El objetivo práctico es evitar anomalías: que corregir un dato obligue a tocar veinte filas, o que borrar un registro se lleve información que no tenía nada que ver.

---

3. Esquemas

3.1 Enunciado resumido

La universidad ofrece **programas**. Cada programa se compone de **cursos**. Los cursos se dictan en **grupos** y cada grupo tiene un **profesor**. Los **estudiantes** se matriculan en grupos, período a período. Cada programa tiene un **plan de estudio**.

3.2 Diagrama entidad-relación

El diagrama usa notación *crow's foot* (pata de gallo). `||` significa "uno y

solo uno", `o{` significa "cero o muchos".

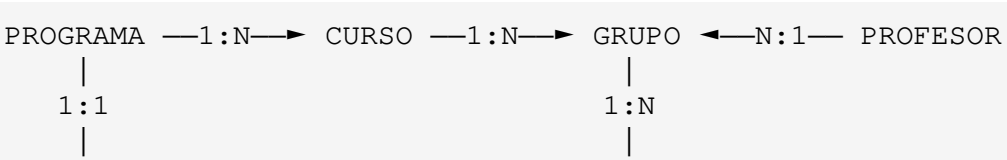
```
erDiagram
    PROGRAMA ||--o{ CURSO : "tiene"
    PROGRAMA ||--|| PLAN_ESTUDIO : "tiene"
    CURSO ||--o{ GRUPO : "se ofrece en"
    PROFESOR ||--o{ GRUPO : "dicta"
    ESTUDIANTE ||--o{ MATRICULA : "realiza"
    GRUPO ||--o{ MATRICULA : "tiene"

    PROGRAMA {
        int id_programa PK
        string nombre
        string nivel
        string facultad
        int duracion
    }
    PLAN_ESTUDIO {
        int id_plan PK
        int id_programa FK
        string nombre_plan
        string descripcion
        date fecha_inicio
        date fecha_fin
    }
    CURSO {
        int id_curso PK
        string nombre
        int creditos
        string descripcion
        int id_programa FK
    }
    PROFESOR {
        int id_profesor PK
        string nombre
        string apellido
        string documento
        string correo
        string telefono
```

```
        string departamento
    }
    GRUPO {
        int id_grupo PK
        int id_curso FK
        int id_profesor FK
        string periodo
        string horario
        string salon
        int capacidad
    }
    ESTUDIANTE {
        int id_estudiante PK
        string nombre
        string apellido
        string documento
        string correo
        string telefono
        date fecha_nacimiento
        string estado
    }
    MATRICULA {
        int id_matricula PK
        int id_estudiante FK
        int id_grupo FK
        date fecha_matricula
        string estado
        decimal nota_final
    }
```

3.3 Vista simplificada de la cadena de dependencias

Leído de izquierda a derecha, cada flecha apunta al lado "muchos":



```
    ▼                                   ▼
PLAN_ESTUDIO                    MATRICULA ◄──N:1── ESTUDIA
```

La entidad `MATRICULA` es el punto donde se encuentran estudiantes y grupos. Todo lo académico del estudiante pasa por ahí.

3.4 Cardinalidades y participación

Relación	Entidades	Cardinalidad	Participación
Tiene	Programa – Curso	1:N	Curso total, Programa parcial
Tiene	Programa – Plan_Estudio	1:1	Total en ambos lados
Se ofrece en	Curso – Grupo	1:N	Grupo total, Curso parcial
Dicta	Profesor – Grupo	1:N	Grupo total, Profesor parcial
Realiza	Estudiante – Matrícula	1:N	Matrícula total, Estudiante parcial
Tiene	Grupo – Matrícula	1:N	Matrícula total, Grupo parcial
(derivada)	Estudiante – Grupo	N:M	Resuelta con Matrícula

---

4. Resolución paso a paso

Paso 1. Subrayar los sustantivos del enunciado

Primer pase de lectura, buscando cosas de las que la universidad necesita guardar datos. Salen siete: **estudiante, programa, curso,**

profesor, grupo, matrícula y plan de estudio. El enunciado además pide exactamente 7 tablas, así que el conteo cuadra.

Una pregunta útil para decidir si algo es entidad o simple atributo: *¿tiene vida propia y características propias?* El "salón" no la tiene (es un dato del grupo). La "matrícula" sí: tiene fecha, estado y nota.

Paso 2. Asignar atributos y escoger claves primarias

El enunciado ya lista los atributos. Lo que hay que decidir es la clave de cada entidad. En todos los casos se usa una **clave sustituta** numérica (`id_*`), que es estable y no cambia aunque un correo o un documento sí lo hagan.

Entidad	Clave primaria	Atributos propios
Estudiante	`id_estudiante`	nombre, apellido, documento, correo, teléfono, fecha_nacimiento, estado
Programa	`id_programa`	nombre, nivel, facultad, duración
Curso	`id_curso`	nombre, créditos, descripción
Profesor	`id_profesor`	nombre, apellido, documento, correo, teléfono, departamento
Grupo	`id_grupo`	periodo, horario, salón, capacidad
Matrícula	`id_matricula`	fecha_matrícula, estado, nota_final
Plan_Estudio	`id_plan`	nombre_plan, descripción, fecha_inicio, fecha_fin

Un detalle que conviene dejar dicho: `documento` y `correo` **no** son la clave primaria, pero sí son **claves candidatas**. Se protegen con `UNIQUE`.

Paso 3. Traducir cada regla de negocio a una relación

Aquí está el corazón del ejercicio. Cada regla del enunciado se convierte en una relación con su cardinalidad. La técnica es leer la regla **en los dos sentidos**.

Regla	Lectura hacia un lado	Lectura hacia el otro	Resultado
1	Un programa tiene muchos cursos	Un curso pertenece a un solo programa	**Programa 1:N Curso**
2	Un curso tiene varios grupos	Un grupo es de un solo curso	**Curso 1:N Grupo**
3	Un profesor dicta varios grupos	Un grupo tiene un solo profesor	**Profesor 1:N Grupo**
4	Un estudiante se matricula en varios grupos	Un grupo tiene varios estudiantes	**Estudiante N:M Grupo**
5	Un programa tiene un plan	Un plan pertenece a un solo programa	**Programa 1:1 Plan_Estudio**
6	Un estudiante pertenece a un único plan	(a través de su programa)	Relación indirecta, ver paso 8

Paso 4. Resolver la relación N:M

La regla 4 genera la única relación muchos a muchos del modelo. Un estudiante cursa varios grupos y un grupo recibe varios estudiantes. En el modelo relacional eso no cabe en dos tablas, así que se introduce **`MATRÍCULA`** en el medio.

El resultado son dos relaciones 1:N:

- **Estudiante 1:N Matrícula** (relación *Realiza*)
- **Grupo 1:N Matrícula** (relación *Tiene*)

`MATRÍCULA` hereda las dos claves foráneas y suma sus datos propios.

Para que un estudiante no quede inscrito dos veces en el mismo grupo, se agrega una restricción de unicidad sobre el par `(id_estudiante, id_grupo)`.

Paso 5. Ubicar las claves foráneas

Regla mecánica: en una relación 1:N, la FK va **del lado N**, apuntando al lado 1.

Relación	Tabla que recibe la FK	FK
Programa → Curso	Curso	`id_programa`
Curso → Grupo	Grupo	`id_curso`
Profesor → Grupo	Grupo	`id_profesor`
Estudiante → Matrícula	Matrícula	`id_estudiante`
Grupo → Matrícula	Matrícula	`id_grupo`
Programa → Plan_Estudio (1:1)	Plan_Estudio	`id_programa` + `UNIQUE`

En la relación 1:1 hay libertad para escoger de qué lado va la FK. Se pone en `Plan_Estudio` porque el plan es el que depende del programa, y así coincide con la lista de atributos del enunciado. El `UNIQUE` sobre `id_programa` es lo que la convierte en 1:1 de verdad. Sin él, la base de datos aceptaría dos planes para el mismo programa.

Paso 6. Esquema relacional resultante

Notación: **subrayado conceptual** en PK con `(PK)`, y `(FK)` con flecha a la tabla referenciada.

```
PROGRAMA       (id_programa PK, nombre, nivel, facultad, d
PLAN_ESTUDIO   (id_plan PK, id_programa FK→PROGRAMA UNIQUE
                descripcion, fecha_inicio, fecha_fin)
CURSO          (id_curso PK, nombre, creditos, descripcion
```

```
                id_programa FK→PROGRAMA)
PROFESOR       (id_profesor PK, nombre, apellido, document
                correo UNIQUE, telefono, departamento)
GRUPO          (id_grupo PK, id_curso FK→CURSO, id_profeso
                periodo, horario, salon, capacidad)
ESTUDIANTE     (id_estudiante PK, nombre, apellido, docume
                correo UNIQUE, telefono, fecha_nacimiento,
MATRICULA      (id_matricula PK, id_estudiante FK→ESTUDIAN
                id_grupo FK→GRUPO, fecha_matricula, estado
                UNIQUE(id_estudiante, id_grupo))
```

Siete tablas, como pedía el enunciado.

Paso 7. Implementación en SQL

Script en sintaxis estándar, probado en lógica para PostgreSQL y fácil de adaptar a MySQL. El orden de creación importa: primero las tablas que no dependen de nadie.

```
CREATE TABLE programa (
    id_programa  INT PRIMARY KEY,
    nombre       VARCHAR(100) NOT NULL,
    nivel        VARCHAR(10)  NOT NULL
                 CHECK (nivel IN ('pregrado', 'posgrado')
    facultad     VARCHAR(100) NOT NULL,
    duracion     SMALLINT     NOT NULL CHECK (duracion >
);

CREATE TABLE plan_estudio (
    id_plan       INT PRIMARY KEY,
    id_programa   INT          NOT NULL UNIQUE,  -- UNIQU
    nombre_plan   VARCHAR(100) NOT NULL,
    descripcion   VARCHAR(255),
    fecha_inicio  DATE         NOT NULL,
    fecha_fin     DATE,
    CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio
    FOREIGN KEY (id_programa) REFERENCES programa (id_pro
);
```

```
CREATE TABLE curso (
    id_curso     INT PRIMARY KEY,
    nombre       VARCHAR(100) NOT NULL,
    creditos     SMALLINT     NOT NULL CHECK (creditos >
    descripcion  VARCHAR(255),
    id_programa  INT          NOT NULL,
    FOREIGN KEY (id_programa) REFERENCES programa (id_pro
);

CREATE TABLE profesor (
    id_profesor   INT PRIMARY KEY,
    nombre        VARCHAR(80)  NOT NULL,
    apellido      VARCHAR(80)  NOT NULL,
    documento     VARCHAR(20)  NOT NULL UNIQUE,
    correo        VARCHAR(120) NOT NULL UNIQUE,
    telefono      VARCHAR(20),
    departamento  VARCHAR(100) NOT NULL
);

CREATE TABLE grupo (
    id_grupo     INT PRIMARY KEY,
    id_curso     INT         NOT NULL,
    id_profesor  INT         NOT NULL,
    periodo      VARCHAR(10) NOT NULL,          -- ej. '
    horario      VARCHAR(60) NOT NULL,
    salon        VARCHAR(20),
    capacidad    SMALLINT    NOT NULL CHECK (capacidad >
    FOREIGN KEY (id_curso)    REFERENCES curso (id_curso)
    FOREIGN KEY (id_profesor) REFERENCES profesor (id_pro
);

CREATE TABLE estudiante (
    id_estudiante     INT PRIMARY KEY,
    nombre            VARCHAR(80)  NOT NULL,
    apellido          VARCHAR(80)  NOT NULL,
    documento         VARCHAR(20)  NOT NULL UNIQUE,
    correo            VARCHAR(120) NOT NULL UNIQUE,
    telefono          VARCHAR(20),
    fecha_nacimiento  DATE         NOT NULL,
```

```
    estado            VARCHAR(15)  NOT NULL
                      CHECK (estado IN ('activo', 'inacti
);

CREATE TABLE matricula (
    id_matricula    INT PRIMARY KEY,
    id_estudiante   INT         NOT NULL,
    id_grupo        INT         NOT NULL,
    fecha_matricula DATE        NOT NULL,
    estado          VARCHAR(15) NOT NULL
                    CHECK (estado IN ('inscrita', 'cancel
    nota_final      NUMERIC(3,1) CHECK (nota_final BETWEE
    UNIQUE (id_estudiante, id_grupo),
    FOREIGN KEY (id_estudiante) REFERENCES estudiante (id
    FOREIGN KEY (id_grupo)      REFERENCES grupo (id_grup
);
```

Los valores de los `CHECK` (estados, escala de notas, unidad de la duración) son **supuestos razonables**: el enunciado no los fija. Cada universidad ajusta los suyos.

Paso 8. Verificar que el modelo cumple las 6 reglas

Se recorre la lista y se localiza en el esquema el mecanismo que impone cada regla.

Regla	¿Cómo la garantiza el esquema?
1	`curso.id_programa` es `NOT NULL` y apunta a un solo programa
2	`grupo.id_curso` es `NOT NULL` y apunta a un solo curso
3	`grupo.id_profesor` es `NOT NULL`, un único profesor por grupo
4	`matricula` cruza estudiante y grupo, con `UNIQUE` sobre el par
5	`plan_estudio.id_programa` es `NOT NULL UNIQUE`

6	Se cumple por derivación, como se explica a continuación

Sobre la regla 6. Dice que un estudiante pertenece a un único plan "a través de su programa". Pero la tabla `ESTUDIANTE` del enunciado **no tiene** una FK hacia programa ni hacia plan. El vínculo se reconstruye por el camino:

```
Estudiante → Matrícula → Grupo → Curso → Programa → Plan_
```

Funciona mientras el estudiante tenga al menos una matrícula. Un estudiante recién admitido, sin ninguna, no pertenece a ningún programa según el modelo. Y nada impide que alguien se matricule en cursos de dos programas distintos.

Para el ejercicio, el modelo cumple lo pedido y se mantiene en 7 tablas con los atributos dados. Si fuera un sistema real, la mejora natural es agregar `id_programa` (FK) a `ESTUDIANTE`, de modo que la pertenencia quede registrada desde la admisión y no se deduzca de las matrículas. Vale la pena plantearla en clase como pregunta de discusión.

Paso 9. Verificar la normalización

Se revisa cada nivel sobre las siete tablas.

1FN. Todos los atributos son atómicos. No hay listas de teléfonos ni columnas repetidas tipo `curso1`, `curso2`. Se cumple.

2FN. Las tablas con clave simple cumplen automáticamente. En `MATRICULA` la clave primaria es `id_matricula` (un solo campo), así que no hay dependencias parciales posibles. Si se hubiera usado la clave compuesta `(id_estudiante, id_grupo)`, la nota y la fecha dependen del par completo, y también se cumpliría.

3FN. Se busca que ningún atributo no clave dependa de otro no clave. Todo atributo describe directamente a su entidad: el `salón` es del grupo, el `departamento` del profesor, los `créditos` del curso. Se cumple.

Hay dos atributos que merecen una nota, porque son los candidatos clásicos a romper la 3FN si el sistema creciera:

- **`facultad`** en `PROGRAMA`: si la facultad tuviera decano, sede o código propio, saldría a su tabla.
- **`departamento`** en `PROFESOR`: mismo argumento, con jefe de departamento o presupuesto.

Con los datos que da el enunciado, ambos son atributos simples que dependen solo de su clave. Convertirlos en tablas aparte sobrepasaría las 7 tablas pedidas sin una necesidad real. Es una decisión de diseño que se justifica, no un descuido.

Paso 10. Pruebas rápidas con consultas

Un modelo se valida consultándolo. Tres ejemplos que recorren las relaciones principales.

Grupos de un profesor, con su curso y período:

```
SELECT p.nombre, p.apellido, c.nombre AS curso, g.periodo
FROM profesor p
JOIN grupo g ON g.id_profesor = p.id_profesor
JOIN curso c ON c.id_curso = g.id_curso
WHERE p.id_profesor = 1;
```

Estudiantes matriculados en un grupo, con su nota:

```
SELECT e.nombre, e.apellido, m.estado, m.nota_final
FROM matricula m
JOIN estudiante e ON e.id_estudiante = m.id_estudiante
WHERE m.id_grupo = 10;
```

Programa de cada estudiante, deducido por sus matrículas (regla 6):

```
SELECT DISTINCT e.id_estudiante, e.nombre, pr.nombre AS p
FROM estudiante e
```

```
JOIN matricula m   ON m.id_estudiante = e.id_estudiante
JOIN grupo g       ON g.id_grupo = m.id_grupo
JOIN curso c       ON c.id_curso = g.id_curso
JOIN programa pr   ON pr.id_programa = c.id_programa
JOIN plan_estudio pl ON pl.id_programa = pr.id_programa;
```

La tercera consulta es la que más ayuda a entender la observación del paso 8: son cinco `JOIN` para responder algo que con una FK directa sería uno.

---

5. Errores frecuentes al resolver este ejercicio

- Poner `id_profesor` en `CURSO`. El profesor se asigna al **grupo**, no al curso.
- Dibujar Estudiante–Grupo como N:M sin entidad intermedia en el modelo relacional.
- Olvidar el `UNIQUE` en el plan de estudio y dejar la relación 1:1 en el aire.
- Confundir cardinalidad con participación: son restricciones distintas.
- Escribir la FK del lado "1" en una relación 1:N.

---

6. Preguntas para discusión en clase

1. ¿Qué cambiaría si un curso pudiera pertenecer a varios programas?
2. ¿Y si un grupo pudiera tener dos profesores? ¿Qué tabla nueva aparece?
3. ¿Dónde se guardaría el historial si un estudiante cambia de programa?
4. ¿Conviene que `periodo` sea una tabla propia? Argumentar en 3FN.

5. ¿Cómo se impediría que las matrículas superen la `capacidad` del grupo?

---

7. Referencias

- Chen, P. P. (1976). *The Entity-Relationship Model: Toward a Unified View of Data*. ACM Transactions on Database Systems, 1(1), 9-36.
- Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377-387.
- Elmasri, R. y Navathe, S. B. *Fundamentals of Database Systems*. Pearson.
- Silberschatz, A., Korth, H. F. y Sudarshan, S. *Database System Concepts*. McGraw-Hill.
- Date, C. J. *An Introduction to Database Systems*. Pearson.